

本科科研预备·初学者实施文档

# 眼在手机械臂 视觉伺服闭环控制 实施文档

一份给控制专业初学者的长篇操作说明书

每一步都写清：为什么做、在算什么、具体怎么做、怎样才算做完

适用：已学 ROS 2 通信、URDF、Gazebo，并看完 MoveIt 课程第八章 Python 调用

不要求：精通 C++、自写驱动、实验室内部模型

仿真模型：课设工作空间中的机械臂与夹爪

真机：组内已有 ROS 接口的 6 轴或 7 轴臂（后半段可选）

## 写给初学者的阅读说明

---

这份文档不是论文，也不是视频课笔记。它是你从“听懂功能包在干什么”走到“自己做出一条视觉控制闭环”的实施说明书。初学者最容易犯的错，不是公式不会写，而是把好几层软件混成一件事：以为 Commander 就是驱动，以为检测一次再规划就是闭环，以为把课看完就等于会做项目。

请按章节顺序做。每一大步都有四个小节：目的、原理、实现、验收。验收没过，不要进入下一步。这样你不会在相机、控制律、启动问题同时爆炸时不知道先查哪一层。

你现在的位置：第一门课已看到第八章（Python 调用 Commander），第二门 ros2\_control 课看得很少。因此本文默认：你会在 Python 里调用规划接口，但还没有自己写完整作业节点。第一步不是加相机，而是把调用做成可重复的脚本。

应用层一律用 Python。你不需要编写 Commander 这个库，也不需要 C++ 重写 MoveIt。你要写的是“调用代码”。实验室臂的 URDF 现在不要拷进仿真。仿真用课设模型即可。

# 第一章 这个项目到底在做什么

---

## 1.1 用生活例子先建立图像

想象你伸手去拿桌上的杯子。一种做法是：先看一眼杯子在哪，闭上眼睛，把手按记忆伸过去。手一动，你就不再看了。这叫开环。杯子若被挪了一点，或你的距离感有误差，手就会偏。

另一种做法是：先大概伸到杯子附近（这一段可以“闭眼走大步”），然后一直盯着杯子和手指的相对位置，手指偏左就往右修，偏远就再靠近，直到对上。这叫闭环。眼睛提供误差，大脑和手臂肌肉按误差不断修正。

本项目里，MoveIt 和 Commander 负责“大概伸过去”的那一大步。相机和 ArUco 标签相当于眼睛，告诉你标签中心偏了图像中心多少像素。你写的控制节点相当于大脑：根据像素误差算出应该往哪边动。ros2\_control 和关节电机相当于肌肉：把速度命令执行下去。下一帧图像变了，误差变了，再算一次。这才是课题组说的视觉—控制闭环：视觉管感知，控制管稳、准、快。

## 1.2 课设抓取和本项目的本质差别

你在课里做的典型流程是：得到一个目标位姿，调用 Commander，MoveIt 规划一条关节轨迹，控制器跟踪这条轨迹直到走完。相机如果用了，也往往只在规划前用一次。运动过程中，图像不再进入控制律。所以它是作业流程，不是视觉伺服科研。

本项目在“走到能看见标签”之后，必须停止规划。此后每一个控制周期都要：读误差 → 算速度 → 发关节速度 → 臂微动 → 新图像。如果你发现伺服阶段还在调用 plan，说明闭环没有建立，项目没有做成。

## 1.3 做完之后你应能向别人讲清的三句话

1. Commander 只负责下目标并请 MoveIt 规划，它不转电机。
2. 轨迹跟踪和速度跟踪发生在 ros2\_control，仿真插件或厂家驱动才是硬件接口。
3. 视觉伺服的贡献在控制律：误差如何变成速度、延迟下为什么会晃、如何限幅和停机。

若这三句还讲不清，说明还停在“会启动功能包”，需要按本文从步骤 A 重新做。

## 1.4 项目分成两段，不要一次做完

| 阶段       | 做什么                              | 算不算科研预备完成         |
|----------|----------------------------------|-------------------|
| 步骤 A 到 E | 启动、调用 Commander、看见标签、开环对照、静态速度闭环 | E 通过即最低完成线，可向组里演示 |
| 步骤 F 到 G | 开环闭环表格；运动标签 + 人为延迟 + 三种控制律       | G 通过即本练习含金量完成     |
| 步骤 H     | 实验室真机复现                          | 可选，须导师同意，占机时间短    |

大三若要投稿，是在 E 或 G 的真机环上再加一个控制问题（延迟稳定或 7 轴），不是把“我对准了 ArUco”写成论文。

## 第二章 你需要具备什么，本文教什么

---

### 2.1 默认你已经会的

- ROS 2 话题、工作空间、colcon、source。
- URDF 是机器人模型，Gazebo 能把模型转起来。
- MoveIt 里有规划组；第八章会用 Python 调用 set\_pose\_target、plan、execute 或 go。
- 机器人学课上的正运动学、坐标系、雅可比名字（允许暂时算不利索，步骤 E 再对着代码补）。

### 2.2 本文会带你做成的

- 自己的 Python 节点调用 Commander，完成到点、下降、开合（若有夹爪）。
- 仿真里眼在手相机和 ArUco 检测。
- 把像素误差变成速度命令，不经 MoveIt。
- 用表格比较开环和闭环；理解延迟为何让环晃。

### 2.3 明确先不要学的

- C++ 重写 Commander，或阅读 MoveIt 规划器源码。
- 第二门课里从零写硬件驱动、写自定义 controller（速度控制器起不来再回头看）。
- YOLO、SLAM、导航、大模型操作。
- 把实验室内部 URDF 拷进自己电脑做仿真。

### 第三章 系统分层：初学者必须建立的地图

把软件想成一条从上到下的管道。上面是“要干什么”，下面是“关节怎么转”。每一层只做一件事。

| 层次    | 日常类比            | 本项目中的软件                      | 初学者易错点                |
|-------|-----------------|------------------------------|-----------------------|
| 任务状态机 | 先走近，再细对，再停下     | 你写的 vs_fsm                   | 把所有逻辑塞进一个脚本，无法判断卡在哪一步 |
| 规划调用  | 在地图上画一条不撞墙的路    | Python 调用 Commander / MoveIt | 以为 go() 就是电机控制        |
| 视觉测量  | 眼睛报告“偏左 20 像素”  | vs_detect + OpenCV ArUco     | 把检测网络当课题，忘记误差要稳定      |
| 控制律   | 偏左就向右修，修多少由增益决定 | vs_servo 里的 image_law        | 增益乱加大，振荡却去改检测器        |
| 关节执行  | 肌肉按速度去跟         | ros2_control 的轨迹或速度控制器       | 控制器不是 active 却怪自己的公式  |
| 仿真或真机 | 真实的手臂           | Gazebo 插件或厂家驱动               | 仿真没通就上真机              |

记住：第八章只训练“规划调用”这一层。ros2\_control 课的前半训练“关节执行”如何启动。视觉伺服的新代码在“测量 + 控制律 + 状态机”。三层都通，项目才立得住。

```

图像 /camera/image_raw
  → vs_detect 算出标签中心和是否看见
  → /vs/error /vs/tag_ok
关节状态 /joint_states 与 TF
  → vs_servo 用误差算速度，再变成关节速度
  → ros2_control 速度控制器
状态机 vs_fsm
  → 接近阶段：调用 vs_commander → MoveIt → 轨迹控制器
  → 伺服阶段：只跑 vs_servo，禁止 plan
  
```

## 第四章 开始之前：课程看到哪，项目从哪开工

---

第一门课《ROS 2 MoveIt 2 - Control a Robotic Arm》看到第六章，模型、MoveIt、bringup 已经够用。第七章是 C++，你可以跳过。第八章是 Python 调用，必须自己敲一遍并跑通。第九章硬件接口先不要看。

第二门课《ROS2 硬件与 ros2\_control》不必现在系统学。只在步骤 A 或 E 发现控制器列表是空的、无法 active 时，再去看“用 Mock Hardware 接上 ros2\_control”那一节。

开工标准：第八章的 Python 例程能让仿真臂走到一个位姿。然后立刻做本文步骤 A 和 B。不要等第二门课看完，不要先加相机。

## 第五章 步骤 A 环境确认与接口清单

---

### 目的

后面每一行代码都要写规划组名字、控制器名字、关节顺序。这些不是通用的，必须从你自己的课设包里抄下来。本步不产生算法，只产生一张清单，避免“抄了手册里的 panda\_arm，自己的组其实叫 arm”。

### 原理

ROS 2 启动后，描述、规划和控制器是三套东西同时活着。描述发布 robot\_description 和 TF；MoveIt 的 move\_group 读规划组；controller\_manager 加载轨迹控制器。任何一层没起来，后面的 go() 都会失败，而且报错不一定好懂。所以先确认三层都活，再写节点。

### 实现（请逐条做，不要跳）

1. 打开终端，执行：source /opt/ros/\$ROS\_DISTRO/setup.bash。若发行版是 Humble，就是 source /opt/ros/humble/setup.bash。
2. 进入课设工作空间：cd ~/workspace\_ws（路径按你本机为准），然后 colcon build --symlink-install，成功后再 source install/setup.bash。
3. 用课设自带的 launch 启动仿真和 MoveIt。命令以你仓库 README 为准，常见类似 ros2 launch xxx\_bringup demo.launch.py。
4. 另开一个已经 source 过的终端，执行下面三条，把输出保存到笔记：

```
ros2 topic list
ros2 topic echo /joint_states --once
ros2 control list_controllers
```

你应当能看到 /joint\_states，以及至少一个名字里带 trajectory 的控制器，状态为 active。

5. 打开 moveit 配置里的 SRDF 或 RViz 的 Planning Group 下拉框，抄下臂的组名、夹爪组名、末端 link、基座 frame。
6. 在 RViz 的 MotionPlanning 插件里拖动末端到一个看起来安全的位置，点 Plan，再点 Execute。仿真里的臂必须真正动起来。

## 请填写的清单

| 项目                    | 你的本机值 |
|-----------------------|-------|
| ROS 发行版               |       |
| 工作空间路径                |       |
| 启动用的 launch 命令        |       |
| 臂规划组名称                |       |
| 夹爪规划组名称（没有就写无）        |       |
| 末端 link 名称            |       |
| 基座 frame 名称           |       |
| 轨迹控制器名称               |       |
| /joint_states 里的关节名顺序 |       |

## 验收

清单填满；RViz 执行成功一次；list\_controllers 里轨迹控制器为 active。任何一项失败，只排启动与依赖，不要开始写伺服。

常见卡点：忘记 source 工作空间；launch 起了 RViz 但没起 controller\_manager；规划组名和代码以后不一致。先把这三件事排除。

## 第六章 步骤 B 用 Python 调用 Commander 做开环流程

---

### 目的

把第八章“会调用”变成你自己的作业脚本：自动走到预对准点，再直线下降一点，若有夹爪就开合。这是整条状态机的骨架。以后视觉只负责提供“预对准点从哪来”，不会推翻这个骨架。

### 原理（请读懂再写代码）

当你调用 `set_pose_target` 再 `go` 时，发生了三件事。第一，Commander 把目标发给 MoveIt。第二，MoveIt 在关节空间搜索一条尽量不碰撞的路径，得到一条随时间变化的关节角轨迹。第三，这条轨迹被发给 `ros2_control` 里的 `joint_trajectory_controller`，控制器每个周期把当前关节角往轨迹上的下一个点推。电机（或仿真里的物理引擎）真正去跟的是第三步。

所以：你会调用，指的是会写第一、第二步的客户端代码。你不会、也不需要编写第三步的控制器和驱动。

笛卡尔直线 `compute_cartesian_path` 略有不同：它先在直线上插很多末端路点，再对每个路点求逆解，拼成关节轨迹。返回值里的 `fraction` 表示直线有多少比例求逆成功。小于 0.95 通常说明中间有奇异、超限或碰撞，不能假装成功。

推荐状态：APPROACH（到预点）→ DESCEND（直线下降）→ GRASP（夹爪，可省略）→ LIFT → DONE。任一步失败进入 ERROR，后面的动作全部取消。初学者必须打印每一步名字，否则失败时不知道卡在规划还是执行。

### 实现

1. 先在 RViz 里用手拖出两个安全位姿：一个略高，相机以后能看到桌面标签；一个比它低几厘米。把位置和姿态抄下来。姿态四元数必须来自你的臂，禁止直接使用本文示例数字。
2. 在工作空间新建功能包 `vs_practice`（或你喜欢的名字），增加 `config/waypoints.yaml`，只存放这些数字。
3. 按第八章的 Python API 写 `vs_commander_node.py`。下面是逻辑示意，类名以你课里为准（MoveItPy、pymoveit2 或课程封装）：

```

arm = create_move_group("这里填清单里的臂组名")
arm.set_pose_target(approach_pose)
success = arm.go(wait=True)
arm.stop()
arm.clear_pose_targets()
if not success:
    打印 ERROR 并退出

# 下降：从当前位姿复制一份，把 z 减少一小段
waypoints = [current_pose, lowered_pose]
traj, fraction = arm.compute_cartesian_path(waypoints, 0.01)
if fraction < 0.95:
    打印笛卡尔失败并退出
arm.execute(traj, wait=True)

```

4. 每一步用 print 或日志输出状态名和成功与否。
5. 另开终端：ros2 bag record /joint\_states /tf，跑一遍后停录，确认 bag 里关节在变。
6. 不碰 RViz 目标，连续自动运行两遍。

## 验收

- 一键（或一条 ros2 run）能走完流程。
- 笛卡尔 fraction 至少 0.95。
- 你能用自己的话说出：谁规划，谁跟踪轨迹。

规划失败：先改接近点，减小下降距离，不要去改 MoveIt 源码。

臂不动：回到步骤 A，看控制器是否 active。

只有 RViz 能拖、脚本不行：多半是规划组名写错，或节点没等到 move\_group 启动。

## 第七章 步骤 C 让仿真“看见”标签

---

### 目的

给系统接上一只眼。本步只测量，不控制。初学者常在这一步就想对准，请忍住：先保证图像话题稳定、检测有开关、坐标系说得清。

### 原理

相机把三维世界压成一张图。ArUco 是印有黑白图案的正方形，检测算法给出四个角点。四个角点的平均位置就是标签中心像素  $(u, v)$ 。图像中心  $(cx, cy)$  来自相机内参。以后的误差就是标签中心减图像中心。

眼在手的意思是：相机固连在末端上，末端一动，相机跟着动。相机相对法兰的安装关系是一个固定变换，仿真里写在 URDF 的 origin 里，运行时出现在 TF 里。这两个必须一致，否则后面手眼连乘会整条链错。

还要注意光学坐标系：常见约定是 Z 沿光轴朝前，X 向右，Y 向下。机械法兰的 Z 往往沿工具朝外。两套轴不一样。把像素误差当成法兰系速度，是闭环反向的最常见原因。

### 实现

1. 在末端 link 下增加 camera\_link。用 Gazebo 相机插件发布 /camera/image\_raw 和 /camera/camera\_info。用 `ros2 topic hz /camera/image_raw` 看帧率，建议 20 到 30 赫兹。
2. 在桌面或地面放一个 ArUco 模型。参数先固定：字典 DICT\_4X4\_50，id 为 0，边长 0.05 米。这些数以后标定和检测必须一致。
3. 写 vs\_detect：订阅图像，用 OpenCV 的 `aruco.detectMarkers`。看见则 /vs/tag\_ok 为真，发布中心像素；看不见为假。图像上画框和十字，用窗口确认。
4. 用 tf2\_echo 查看末端到 camera\_optical\_frame 的变换，对照 URDF。

### 验收

看见标签时 tag\_ok 为真，手挡住为假。平移标签，像素明显变化。本步仍然禁止把结果送给 MoveIt。

## 第八章 步骤 D 开环视觉引导（对照用的基线）

---

### 目的

让“看见”第一次影响运动：根据当前图像算一个接近位姿，调用步骤 B 的 Commander 走过去。同时故意保持开环——中途不再看——以便和后面的闭环对比。

### 原理

若检测还能给出标签在相机坐标系下的位姿，则基座下的标签位姿为：从基座到末端，再乘末端到相机，再乘相机到标签。预对准点取在标签前方、沿视线后退约 0.2 米，让相机正对标签。这个位姿只在接近开始时算一次。执行过程中即使你把标签挪开，轨迹仍走向旧目标。这正是开环的定义，不是缺陷；步骤 F 要的就是这个对照。

如果暂时只有像素、没有可靠三维位姿，可以用“假定深度 + 像素偏移”估一个很粗的目标。精度差没关系，因为步骤 E 不再依赖这个估的位姿，而改用纯像素误差。

### 实现

1. 状态机在 APPROACH 入口等待 tag\_ok，取一次测量，计算 approach\_pose，调用 go。
2. 打印当时的像素和目标位置。
3. 到位后再读一次像素，记为开环终值误差，留给步骤 F。
4. 挪动标签再跑一次，确认新目标会变；单次执行中途不要重新 plan。

### 验收

你能对别人说：“图像只用了一次。”保存 bag。本关若已经把误差送进速度环，说明你跳步了，应拆开。

## 第九章 步骤 E 静态视觉速度闭环（项目的心脏）

---

### 目的

这是本项目与课设的分界。规划器退出。误差每个周期都要变成速度。做完这一步，你才真正碰了视觉—控制闭环。

### 原理（请慢慢读）

误差。取图像中心为期望点。 $e_u = u - cx$ ,  $e_v = v - cy$ 。误差是像素，不是米。初学者先不要把像素硬换成米，除非你已经会用内参和深度。

最简单的控制律。标签在画面右边（ $e_u$  为正），相机应向右跟上，使标签回到中心。写成速度： $v_x = -\lambda \cdot e_u$ ,  $v_y = -\lambda \cdot e_v$ 。 $\lambda$  是正的增益。 $\lambda$  太小，爬得很慢； $\lambda$  太大，会冲过中心再折回来，看起来像抖动。调参顺序只能是：先小到刚能动，再慢慢加，出现抖动就回调。

$$v_x = -\lambda \times e_u, \quad v_y = -\lambda \times e_v, \quad \text{转动先取 } 0$$

更接近教科书的写法。用焦距和假定深度把像素误差缩放到大致的米/秒，仍是比例律，只是单位更合理。步骤 E 用哪一种都可以，但必须固定一种，并写在笔记里。

从相机速度到关节速度。机械臂不能直接执行“相机向右 2 厘米每秒”。要用雅可比把笛卡尔速度映射成各关节角速度。近奇异时伪逆会爆，加上阻尼。算出来的关节速度必须限幅，否则仿真会飞、真机会危险。

为什么伺服阶段不能再 plan。plan 生成的是一整段未来轨迹，它不会根据下一帧误差改方向。你若边伺服边规划，等于一只手闭环、一只手开环抢方向，环是乱的。

停机。误差连续很多周期都很小，进入保持，速度发零。连续很多帧看不见标签，也必须发零并中止。这是安全，不是附加题。

### 实现

1. 查清单：仿真是否提供速度接口。若有，伺服前把轨迹控制器切到速度控制器。命令形态如下，名字换成你的：

```
ros2 control switch_controllers \
  --activate joint_group_velocity_controller \
  --deactivate 你的轨迹控制器名
```

若完全没有速度接口，先停下来查课设和官方包，不要用很低频率的位置点假装闭环。

2. 写 vs\_servo，用定时器，周期约 0.02 到 0.05 秒。伪代码：

```
若看不见标签：
    丢失计数加一
    发布全零速度
    若丢失太多：通知状态机中止
    返回
丢失计数清零
e = 当前像素 - 图像中心
V = image_law(e, λ)      # 第 G 步只改这一行
qdot = 阻尼伪逆(J, V)
把 qdot 限制在最大速度内
发布 qdot
```

3. 状态机：APPROACH 成功后切换控制器，进入 SERVO。HOLD 或 ABORT 后再切回轨迹控制器，方便下一次规划。
4. 调  $\lambda$ ：只改参数。不要为了“看起来稳”去改检测阈值来骗自己。
5. 做三组初始偏置：标签在画面左、右、上，看能否回到中心。

## 验收

- 伺服日志里没有 plan。
- 误差能缩小；若越修越偏，先改坐标轴符号，不要先加  $\lambda$ 。
- 挡住标签，速度命令变成 0。
- 你能写出：e 在哪个坐标系，速度在哪个坐标系，雅可比的列对应哪些关节。

步骤 E 通过，就是进组演示的最低完成线。到这里再考虑加延迟和三种律。

## 第十章 步骤 F 用数字比较开环和闭环

---

### 目的

课题组说稳、准、快。没有数字，就只是感觉。本步强迫你用同一初始条件对比步骤 D 和步骤 E。

### 原理

开环的终点误差主要来自标定、规划和控制器跟踪公差，过程中没有纠正。闭环会在过程中减小误差，但增益大了会振荡，于是“快”和“稳”打架。公平比较的前提是：同一初始关节、同一标签、同一相机、同一超时定义。

### 实现

1. 用参数 mode:=open 或 closed 跑两种模式。
2. 每种至少 5 次。超时、看丢、超限、碰撞都记失败，不要删失败数据。
3. 从 bag 计算：最后 0.5 秒平均像素误差（准）；第一次进入阈值并保持 0.3 秒的时间（快）；误差来回穿越或峰值次数（稳）。
4. 出表和误差曲线。禁止用眼睛估计填数。

### 验收

有表、有曲线、有一句老实的结论。例如：“闭环更准，但有一次因增益偏大出现振荡。”没有表，本步不算完成。

## 第十一章 步骤 G 给初学者加一点真正的控制思考

---

### 目的

静态对准会了之后，再让目标慢慢动，并故意让图像晚到。你会看到：同样的比例律，在延迟下可能抖动。这不是再做一个视觉功能，而是让你体会离散控制和过时反馈。这是本练习的含金量，也是以后大三题目的种子。

### 原理

图像晚到 100 毫秒，控制律以为现在还偏很多，继续加油。等图像赶上时，臂已经走过头，于是反向再冲，形成振荡。把增益按延迟减小，相当于不敢踩那么深的油门，通常更稳，但可能更慢。用上一拍误差变化去“猜”当前误差，有时能减小滞后，猜错了会更晃。三种律必须共用同一套检测、限幅和停机，只替换算速度的那一个函数。否则比较不公平。

### 实现

1. 先确认步骤 E 在静止、无延迟时已经稳。
2. 让标签以大约 1 到 2 厘米每秒慢速移动，不要瞬移。
3. 在检测和控制之间加队列，用参数 `image_delay_ms` 取出过时误差。
4. 三种律：固定  $\lambda$ ； $\lambda$  随延迟变小；用差分预测误差后再乘  $\lambda$ 。
5. 实验格子：静止无延迟；静止加 50、100、150 毫秒；慢速运动加同一组延迟。每格多次重复。
6. 写下：预测律在哪一档延迟上不如降增益。

### 验收

参数能在 `launch` 里复现。你能用“过时误差还在加油门”解释振荡，而不是只说“我把  $\lambda$  调小就好了”。

## 第十二章 步骤 H 真机（可选，且必须先批准）

---

目的不是换算法，而是看真实延迟和手眼误差会不会把步骤 E 打坏。仿真仍然用课设模型。真机用实验室官方 ROS 包，不要把课设里的 hardware 源码抄到真机上。

实现顺序：问导师臂是 ROS 1 还是 ROS 2、驱动包名、法兰能否固定相机、急停怎么用 → 官方例程使能并慢速直线运动 → 打印标签、手眼标定并记录残差 → 限速比仿真更保守，复现步骤 E → 有时间再做 F。步骤 G 可用手缓慢挪标签纸，延迟继续用软件注入。全程录 bag。结束松使能。增益仍回仿真里调。

验收：真机能对中，或能解释失败来自标定和接口，而不是逻辑抄错。占机要短、要集中。

## 第十三章 初学者排错顺序

出问题，按这个顺序查，不要同时改三个地方。

| 现象         | 先查                                 | 先不要做        |
|------------|------------------------------------|-------------|
| 完全不动       | 是否 source、launch 是否完整、控制器是否 active | 改控制公式       |
| 只有 RViz 能拖 | 规划组名、move_group 是否已启动、脚本是否等到服务     | 重装 ROS      |
| 规划总失败      | 目标是否在碰撞或超限，下降是否太深                  | 改 MoveIt 源码 |
| 看不见图       | 相机话题、Gazebo 插件、帧率                  | 上 YOLO      |
| 检测有、走反了    | 光学系和误差符号                           | 加大增益        |
| 对准时抖动      | $\lambda$ 、是否限幅、图像是否已经很晚           | 换检测器        |
| 伺服时又在规划    | 状态机是否还调用 go/plan                   | 继续加路点       |

## 第十四章 进度表与每周检查

| 步骤                | 状态  | 完成日期 | 备注        |
|-------------------|-----|------|-----------|
| A 环境清单            | 未开始 |      | 第八章跑通后立刻做 |
| B 调用 Commander 流程 | 未开始 |      |           |
| C 检测              | 未开始 |      |           |
| D 开环视觉            | 未开始 |      |           |
| E 静态闭环（最低完成线）     | 未开始 |      |           |
| F 对照表             | 未开始 |      |           |
| G 运动与延迟（含金量）      | 未开始 |      |           |
| H 真机              | 未开始 |      | 须批准       |

每周问自己三句：有没有新的可运行实验？视觉有没有滑去搞检测网络？过了 E 之后，能不能用曲线解释一次失败？连续两周答不上来，就停止加功能。

向协助者汇报时请带：当前步骤、改了哪个节点、终端关键几行、成功还是失败。不必每次传整个工作空间。

## 第十五章 名词表（初学者对照）

| 词            | 在本项目里的意思                    |
|--------------|-----------------------------|
| Commander    | Python 里调用 MoveIt 的客户端，不是驱动 |
| MoveIt       | 自由空间规划器，伺服阶段必须退出            |
| ros2_control | 关节控制器框架，跟踪轨迹或跟踪速度           |
| hardware 接口  | 仿真插件或厂家驱动，read/write 关节     |
| 开环           | 看一次或规划一次，过程中不再用图像改速度        |
| 闭环           | 每一帧误差都进入控制律                 |
| 眼在手          | 相机装在末端，随臂运动                 |
| ArUco        | 便于稳定检测的二维码式标签               |
| 增益 $\lambda$ | 误差到速度的比例，太大振荡，太小太慢          |
| 雅可比          | 笛卡尔速度与关节速度之间的线性映射           |

本文是实施文档，不是投稿论文。完成步骤 E 即可向课题组演示闭环能力。完成步骤 G 再谈含金量。仿真用课设臂，真机用组内 ROS 臂。第一门课第八章之后开始步骤 A、B。